

实现路线图

以下是分阶段实现 Beethoven 项目的路线图。每一阶段都有明确目标、可交付成果和衡量标准，按难度递增。

核心原则： - **先跑通，再优化**——第一个版本无论多粗糙，跑通整个 pipeline 比追求 SOTA 重要 - **先听懂，再写代码**——每进入一个新阶段，先运行现有工具（Demucs、Spleener 等你的补充）感知结果 - **能听、能看、能量化**——每个阶段结束后，你能"听"到分离结果、"看"到频谱图、"量"到 SDR 分数

阶段 0：环境搭建和第一次"听觉震撼"（1-2 天）

目标：让现有的 SOTA 模型在你的电脑上运行，体验声源分离"能做到什么程度"。

一步安装 Demucs（Meta 的 SOTA 模型）

```
pip install demucs
```

音乐多乐器声源分离的技术构想

在音频与音乐信号中，一首完整的乐曲通常由多种乐器的声波信号叠加融合而成。混合音频的波形本质是各类乐器独立声波的叠加，由此可提出猜想：可通过计算机技术对混合音频波形进行逆向拆解，实现不同乐器声源的独立提取与还原。

从信号处理角度分析，连续的音乐波形承载了全部的声音信息。为实现声源拆解，可将整段时长的音频划分为若干极短的时间微元片段，对每一个时刻、每一段短时波形进行独立求解，拆分出该时刻对应的各类乐器基础声波分量，最终得到全时段的声源分解结果。

该拆解方式存在核心问题：单一混合波形对应无数组波形分解解，属于典型的多解问题，单纯数学拆解无法得到贴合真实演奏的有效结果。因此，需要为求解过程增设物理与音乐层面的约束条件，筛选合理最优解。

核心约束为时序连续性约束：真实乐器的演奏具备自然连续性与规律性，单一乐器在相邻时刻的声波形态、音量、音色不会出现突兀跳变。因此各时刻拆解得到的乐器波形解，必须与前后时刻的分解结果形成时序呼应、平滑衔接，剔除违背自然演奏规律的无效解。

在此基础上，可引入人工智能模型优化求解过程。通过让模型海量学习乐谱数据与常规作曲、演奏规律，习得音乐底层逻辑与乐器发声特征，以此作为判别依据，校验波形分解结果的合理性。通过信号拆解、时序约束、乐理先验校验的结合方式，能够大幅提升多乐器声源拆解的准确率，实现乐曲乐器声部与乐谱信息的高精度还原。

我才大一，懂得不多，以上内容皆为我一天晚上洗澡时所想交给一个AI用更专业的口吻改写而成

接下来我希望能给我一份设计方案，来一步步实现这个技术。

另外我还有另一个思路，根据b站一些鬼畜视频，某些up主用一个乐器，比如钢琴，就可以还原名场面“鸡汤来了”。但是钢琴一个乐器不合适还原全部。同理，我也可以对一个音频文件用多个乐器进行多次这样的操作，然后删去不符合他的片段的的声音，把所有乐器的声音再拼起来。也是另外一个思路，我把这一段称为二号思路，可能比较草率和不清楚

我的作品的名字我想取名beethoven，请把我上面的思路写到D:\zeroC\test\beethoven这里，文档名字你来取，顺便你也加上你的理解，并且写上一个大致实现的思路和设计，我比较熟悉Python语言，语言的选取你可以有自己的想法，我都会听你的有道理的意见

对一首歌做分离

```
demucs --two-stems=vocals your_song.mp3
```

输出在 ./separated/your_song/vocals.wav 和 no_vocals.wav

你在这里需要感受的： - 分离后的人声有多干净？ - 乐器轨听起来有什么artifact？ - 什么音乐分离效果好/差？为什么？

同时建议：

```
pip install librosa matplotlib soundfile torch torchaudio
```

听一首歌的分离结果，就是最好的入门教程。你会立刻在脑海中形成"分离质量的上限在哪里"这个直觉——这对之后你写自己的模型非常有用。

阶段 1：合成数据 + 小实验 (2-3 周)

目标：用可控的合成数据验证"从混合信号中分离乐器"这个想法在数学上可行。

Step 1.1 合成多乐器数据

```
# 概念脚本: synthesize_data.py
goal = "给定一个 MIDI 文件，渲染为 3 个不同乐器的"湿"音频（钢琴、小提琴、长笛）"
steps = [
    "1. 用 music21 或 midi 库解析 MIDI 文件",
    "2. 用 FluidSynth + GeneralUser GS 音源库渲染为每个乐器独立音轨",
    "3. 导出: mix.wav, piano.wav, violin.wav, flute.wav",
]
```

推荐使用的库： - [music21](#)：MIT 开发的音乐分析库，处理 MIDI/乐谱很方便 - [pyfluidsynth](#)：FluidSynth 的 Python 绑定，可以从 MIDI 合成音频 - **音源库 (SoundFont)**：推荐 [GeneralUser GS](#) 或 FluidR3_GM——免费、音色多

Step 1.2 用线性方法建立 Baseline

在深度学习之前，先用经典信号处理方法做分离，你会立刻理解"为什么需要深度学习"：

```
from scipy.signal import wiener

# 维纳滤波：假设乐器在不同频带
# 效果：对非重叠频率的乐器勉强可用，一混叠就彻底失效

from sklearn.decomposition import NMF

# 非负矩阵分解，将频谱分解为"频谱模板 x 时间激活"
# 效果：对频谱结构简单的乐器（如钢琴、吉他）比维纳滤波好
# 但在复杂乐器（如弦乐组）上失效
```

关键认知要在这个阶段获得： - 当两个乐器频率范围不重叠（如贝斯 vs. 三角铁）→ 容易分 - 当两个乐器频率范围重叠（如两把吉他）→ 纯信号处理无法解决 → **需要深度学习**

Step 1.3 验证你的"时序连续性"直觉

在合成数据上，用以下方式验证你的核心想法：

```
# 创建"违反时序连续性"的分离结果（每帧独立随机分离）
# → 听感会有很多"咔嚓"爆音和跳跃
```

```
# 添加时序平滑（相邻帧取平均）
# → 听感明显改善
```

这个小小的实验会让你深刻理解为什么时序连续性约束如此重要。

交付物

成果清单：

- `synthesise_data.py` — 合成多乐器混合+独立轨的脚本
- `baseline_nmf.py` — NMF 分离实现
- `baseline_eval.py` — SDR/SIR 计算脚本
- 实验笔记：什么乐器组合容易分、什么困难

阶段 2：第一个深度学习模型——频谱 U-Net (4-6 周)

目标：实现你自己的第一个声源分离神经网络，在合成数据上产生有意义的结果。

Step 2.1 数据管线

构建能够高效加载数据的 PyTorch Dataset：

```
class InstrumentSeparationDataset(Dataset):
    """返回 (mixture, 各乐器频谱) 对"""
    def __init__(self, root_dir, instruments=['piano', 'violin', 'flute']):
        # 扫描目录，找到所有 mix.wav/stem_x.wav
        # 在 __getitem__ 中：加载 → STFT → 振幅谱 (log scale)
        pass

    def __getitem__(self, idx):
        # 返回：
        #   mix_spec: (1, F, T) 混合频谱
        #   target_specs: (C, F, T) 各乐器的频谱
        pass
```

数据增强（很重要）：对合成数据进行随机增益、微调速度、添加混响，提升泛化能力。

Step 2.2 实现 SimpleU-Net

从 `approach_1.md` 中的骨架代码出发，参照以下具体建议：

```
class SimpleUNet(nn.Module):
    """
    实现细节：
    - 输入: (B, 1, F, T) 混合振幅谱
    - 输出: (B, C, F, T) 每个乐器的掩码

    关键参数：
    - n_fft=1024, hop_length=256 → F=513 频率 bins
    - sr=22050, 每帧约 12ms
```

```
"""
# ... (参见 approach_1.md 中的代码骨架)
```

训练要点: - **Loss 函数:** L1 Loss 在频谱域 + 可选的时序连续性 Loss - **验证:** 每个 epoch 结束后, 对一首验证歌曲做分离, 用耳朵听 - **Batch size:** 从 4 开始, 根据 GPU 显存调整 - **学习率:** 1e-4 (Adam 优化器)

Step 2.3 训练循环

```
for epoch in range(100):
    for batch in dataloader:
        mix, targets = batch
        masks = model(mix)          # 预测掩码
        predictions = masks * mix    # 应用到混合频谱

        loss_l1 = F.l1_loss(predictions, targets)
        loss_tc = temporal_continuity(masks)
        loss = loss_l1 + 0.1 * loss_tc

        loss.backward()
        optimizer.step()

# 每个 epoch 后评估
sdr = evaluate(model, validation_set)
print(f"Epoch {epoch}: L1={loss_l1:.4f}, SDR={sdr:.2f}dB")
```

你的第一次关键瓶颈: 训练初期 loss 下降很慢, 别灰心——这是正常的。当 loss 开始下降后, 分离质量的"跳跃"会很突然 (通常在 20-30 epoch 后)。

Step 2.4 从合成数据迁移到真实音乐

在合成数据上有了较好的分离效果后: 1. 下载 MUSDB18 数据集的子集 (50 首歌曲) 2. 把你的模型在真实数据上 fine-tune 3. 对比合成数据训练 vs. 真实数据训练的差异

交付物

成果清单:

- dataset.py - 数据加载与预处理
- model_unet.py - SimpleU-Net 实现
-
- train.py - 训练脚本
- eval.py - SDR 评估脚本
-
- 实验笔记:
 - 合成数据上的 SDR 分数 (钢琴/小提琴/长笛分离)
 - loss 曲线截图
 - 分离音频样本 (上传到网上方便分享)

阶段 3: 工程化与质量提升 (4-8 周)

目标：提升分离质量，向 Demucs 级别靠拢。

Step 3.1 从合成数据切换到真实数据

MUSDB18 的 150 首歌曲是你这个阶段的主要数据集。你需要解决： - 乐器类别从合成数据的"指定乐器"变为"人声/贝斯/鼓/其他"（MUSDB18 的四轨格式） - 真实录音的混响、噪声、动态范围挑战

Step 3.2 升级模型架构

尝试以下改进（按优先级排序）：

```
# ① 增加通道数（16→32→64 vs 8→16→32）
# ② 加入残差连接（ResBlock）
# ③ 加入 Transformer 层
#      ┌ Encoder 输出 → Transformer → Decoder ┐
#      混合频谱 ─────────────────────────────────→ 掩码
# ④ 使用复数频谱（同时学习振幅和相位）
```

Step 3.3 实现构想二模块

开始探索你的"逐乐器重演"思路：

```
# 1. 用预训练的 AMT 模型（如 Basic Pitch）检测音符
pip install basic-pitch

# 2. 将检测到的音符用高质量音源重新合成
# 3. 用你阶段 2 训练的掩码模型作为"置信度评估器"
# 4. 两方法融合（见 approach_2.md 第 6 节）
```

这个阶段你可以写一个 `ensemble.py`，实现构想一和构想二的融合。

交付物

成果清单：

- `model_improved.py` - 升级版模型架构
- `train_musdb.py` - 在 MUSDB18 上训练的脚本
-
- `transcription.py` - AMT + 合成（构想二路径A）
- `ensemble.py` - 两种构想的融合模块
-
- `results/` - 分离音频样本
- SDR：人声 ~5-7dB，贝斯 ~3-5dB
 (Demucs 约 7-10dB，以此为参照但不苛求)

阶段 4：探索与定制（持续）

目标：做出你自己的贡献——无论是新架构、自定义乐器分离、还是有趣的应用。

你特有的优势

- **两种思路的结合**：目前学术界要么做掩码法，要么做生成法，很少有人系统地融合两者。这就是你可能的创新点。
- **你能控制的数据**：如果你喜欢某个特定乐器（如钢琴、古筝），你可以专门优化针对该乐器的分离——小而专注的模型往往比通用大模型在特定场景表现更好。
- **非西方音乐**：目前大多数数据集和模型基于西方音乐。如果你感兴趣民族乐器分离（二胡、琵琶、古筝），这是一个有意义的探索方向——而且需求很强（很多音乐人想从老录音里分离民族乐器）。

探索方向

方向 1：自定义乐器分离

训练一个"钢琴+吉他专用"的二轨分离模型

方向 2：可视化工具

把你的分离过程做成网页应用，用户上传歌曲→选择乐器→听到分离结果

用 gradio 或 streamlit 一天内就能做出来

方向 3：MIDI 输出

不仅分离乐器，还输出每件乐器的 MIDI 乐谱——"听感识别谱面"

这对音乐教育很有价值

方向 4：实时分离

用模型量化（torch.quantization）和 ONNX Runtime 加速

目标是实现实时/低延迟分离

总体时间线

Day 1-2: 阶段 0 – 环境搭建、跑通 Demucs、获得"听觉震撼"

↓

Week 1-3: 阶段 1 – 合成数据、线性 baseline、验证核心想法

↓

Week 4-9: 阶段 2 – 第一个深度学习模型（U-Net）+ 在合成数据上跑通

↓

Week 10-17: 阶段 3 – 真实数据 + 架构升级 + 构想二融合

↓

Week 18+: 阶段 4 – 探索、定制、贡献

总时间：4-6 个月以业余时间投入，或 2-3 个月全情投入。

每个阶段结束后，你必然会有新的想法和见解——这就是你调整路线图的时机。

硬件需求

阶段	最低配置	推荐配置	替代方案
0-1	任意电脑	任意电脑	—
2	4GB GPU 显存	8GB+ (RTX 3070/4060)	用 CPU 小 batch 训练（慢 5-10 倍）
3	8GB GPU 显存	16GB+ (RTX 4090)	Google Colab（免费/付费）

阶段	最低配置	推荐配置	替代方案
4	取决于探索方向	—	—

没有 GPU 怎么办？ - **Phase 0-2**: 完全可以在 CPU 上跑，只是训练慢 - **Phase 3**: 推荐使用 [Google Colab](#) (免费提供 T4 GPU，可以满足 MUSDB18 训练)。你只需要一个浏览器和 Google 账号。

第一个 Checkpoint: 我应该继续吗？

在完成**阶段 1**后，问自己三个问题：

- 1. **我能从合成音频中分离出不同乐器吗？**（即使效果很一般）
- 2. **我理解了"为什么纯信号处理不够，需要深度学习"吗？**
- 3. **我觉得这个过程有趣吗？**

如果三个都是"是"——继续，你会在声源分离上做得很好。如果第三个是"否"——音频可以换一个方向（如音乐生成、乐器识别、编曲推荐），不必强求分离。

最后一条建议：**公开分享你的进展**。把你的合成数据、第一次分离结果（即使很糟糕）、SDR 分数发在 GitHub 或知乎上。一方面别人会给你反馈和鼓励，另一方面这也能帮到你后续做学院工作时的简历。